

SynaptiQ ResearchOS — Source Code Submission

Capgemini Agentify AI Buildathon 2026 · Round 2 Deep Dive

Repository: [ADD YOUR GITHUB URL HERE]

Live demo: [ADD VERCEL URL WHEN DEPLOYED] → API on Render

Generated: 2026-06-11 22:34 UTC

This PDF summarizes the SynaptiQ codebase structure and includes representative excerpts from the multi-agent pipeline, citation integrity, benchmark evaluation, API routes, frontend client, and Docker stack.

Repository Structure

```
synaptiq/
├── backend/app/agents/
│   ├── __init__.py
│   ├── base.py
│   ├── brief_agent.py
│   ├── comparative_agent.py
│   ├── discovery_agent.py
│   ├── gap_agent.py
│   └── verification_agent.py
├── backend/app/graphs/
│   ├── checkpointer.py
│   ├── research_graph.py
│   ├── routing.py
│   └── state.py
├── backend/app/services/
│   ├── __init__.py
│   └── ...
├── backend/app/api/
│   ├── __init__.py
├── frontend/app/
│   ├── layout.tsx
│   └── page.tsx
├── frontend/components/
│   ├── AgentTimeline.tsx
│   ├── AiMeshBackground.tsx
│   ├── AnalysisOverlay.tsx
│   ├── BriefViewer.tsx
│   ├── ContradictionPanel.tsx
│   ├── ErrorBanner.tsx
│   ├── EvidenceExplorer.tsx
│   ├── GraphViewer.tsx
│   ├── QueryConsole.tsx
│   └── StatsBar.tsx
├── docker/
│   ├── docker-compose.yml
│   └── otel-collector.yaml
├── deployment/azure/
│   ├── main.bicep
│   └── README.md
├── scripts/
│   ├── generate_ppt_content_pdf.py
│   └── generate_source_code_pdf.py
├── README.md
├── ARCHITECTURE_BLUEPRINT.md
└── SUBMISSION.md
```

backend/app/graphs/research_graph.py

```
"""LangGraph research pipeline with discovery through report generation."""

from __future__ import annotations

import uuid
from typing import Any, Awaitable, Callable

from langchain_core.runnables import RunnableConfig
from langgraph.graph import END, START, StateGraph

from app.agents.brief_agent import BriefAgent
from app.agents.comparative_agent import ComparativeAgent
from app.agents.discovery_agent import DiscoveryAgent
from app.agents.gap_agent import GapAgent
from app.agents.verification_agent import VerificationAgent
from app.core.runtime_paths import get_graphs_dir
from app.graphs.checkpointer import get_checkpointer
from app.graphs.routing import route_after_discovery, route_after_verification
from app.graphs.state import ResearchState
from app.models.enums import AgentName
from app.schemas.knowledge_graph import KnowledgeGraphSnapshot
from app.services.agents.observability import persist_agent_log
from app.services.embeddings.embedding_pipeline import EmbeddingPipeline
from app.services.kg.graph_builder import KnowledgeGraphBuilder
from app.services.kg.visualizer import KnowledgeGraphVisualizer
from app.services.reports.report_service import ReportService

NodeFn = Callable[[ResearchState, RunnableConfig], Awaitable[dict[str, Any]]]

def _merge_errors(state: ResearchState, result: dict[str, Any]) -> list[dict[str, Any]]:
    return list(result.get("errors") or [])

async def _persist_log(
    state: ResearchState,
    config: RunnableConfig,
    result: dict[str, Any],
) -> None:
    session_id = state.get("session_id")
    db_session = (config.get("configurable") or {}).get("db_session")
    if session_id and db_session and result.get("agent_log"):
        await persist_agent_log(
            db_session,
            session_id=session_id,
            agent_result=result,
        )

async def node_discovery(state: ResearchState, config: RunnableConfig) -> dict[str, Any]:
    """Run the discovery agent and merge outputs into state."""
    query = state.get("query") or state.get("question") or ""
    filters = state.get("filters") or {}
    control = state.get("control") or {}
    iteration = int(control.get("iteration", 0)) if isinstance(control, dict) else 0
    known_ids = {paper["paper_id"] for paper in state.get("papers", [])}

    pipeline = EmbeddingPipeline()
    agent = DiscoveryAgent(embedding_pipeline=pipeline)
    try:
        result = await agent.run(
            query=query,
            filters=filters,
            known_paper_ids=sorted(known_ids),
            iteration=iteration,
        )
    finally:
        await agent.close()

    if not result.get("papers"):
        from app.services.benchmark.curated_corpus import load_curated_papers
        from app.services.sources.paper_retrieval import PaperRetrievalService

        rescue = PaperRetrievalService()
        try:
            rescued, _meta = await rescue.search(
                query,
                limit=int(filters.get("max_papers", 8)),
                known_paper_ids=known_ids,
            )
        )
```

... (research_graph.py truncated at 80 lines)

backend/app/graphs/routing.py

```
"""Conditional routing for the research graph."""

from __future__ import annotations

from typing import Literal

from langgraph.graph import END

from app.graphs.state import ResearchState
from app.models.enums import Sufficiency

def route_after_discovery(
    state: ResearchState,
) -> Literal["discovery", "verification", "__end__"]:
    """Route after discovery based on sufficiency and iteration budget."""
    control = state.get("control") or {}
    sufficiency = control.get("sufficiency")
    iteration = int(control.get("iteration", 0))
    max_iterations = int(control.get("max_iterations", 2))
    papers = state.get("papers") or []

    if not papers:
        return END

    if sufficiency == Sufficiency.INSUFFICIENT.value and iteration < max_iterations:
        return "discovery"
    return "verification"

def route_after_verification(
    state: ResearchState,
) -> Literal["discovery", "comparative"]:
    """Route after verification; loop to discovery if claims are too thin."""
    verified_claims = state.get("verified_claims") or []
    control = state.get("control") or {}
    iteration = int(control.get("iteration", 0))
    max_iterations = int(control.get("max_iterations", 2))
    unsupported_ratio = float(control.get("unsupported_ratio", 0.0) or 0.0)
    errors = state.get("errors") or []
    verification_failed = any(error.get("agent") == "verification" for error in errors)

    if verification_failed and not verified_claims:
        return "comparative"
    if len(verified_claims) >= 2:
        return "comparative"
    if iteration >= max_iterations:
        return "comparative"
    if unsupported_ratio > 0.8:
        return "discovery"
    if len(verified_claims) < 2:
        return "discovery"
    return "comparative"
```

backend/app/agents/discovery_agent.py

```
"""Discovery agent orchestrating sources, LLM judging, and retrieval."""

from __future__ import annotations

import json
from typing import Any

from sqlalchemy.ext.asyncio import AsyncSession

from app.agents.base import BaseAgent
from app.core.discovery_config import get_discovery_config
from app.core.exceptions import AgentExecutionError, FabricationError, SchemaValidationError
from app.services.benchmark.curated_corpus import load_curated_papers
from app.core.retry import async_retry
from app.database.repositories.paper_repository import PaperRepository
from app.models.enums import AgentName, DiscoveryStatus, JobStatus, Sufficiency
from app.prompts.loader import load_prompt
from app.schemas.agent_io import ControlState, DiscoveryOutput, DiscoveryPaperOutput
from app.schemas.common import PaperRef
```

```

from app.services.embeddings.embedding_pipeline import EmbeddingPipeline
from app.services.llm.base import LLMClient
from app.services.llm.factory import get_llm_client
from app.services.retrieval.retrieval_pipeline import RetrievalPipeline
from app.services.sources.paper_retrieval import PaperRetrievalService

class DiscoveryAgent(BaseAgent):
    """Discover relevant papers and prepare retrieval artifacts."""

    agent_name = AgentName.DISCOVERY

    def __init__(
        self,
        *,
        retrieval_service: PaperRetrievalService | None = None,
        llm_client: LLMClient | None = None,
        embedding_pipeline: EmbeddingPipeline | None = None,
        retrieval_pipeline: RetrievalPipeline | None = None,
        db_session: AsyncSession | None = None,
        max_attempts: int = 3,
    ) -> None:
        super().__init__(max_attempts=max_attempts)
        self._retrieval_service = retrieval_service or PaperRetrievalService()
        self._llm = llm_client or get_llm_client()
        self._embedding_pipeline = embedding_pipeline or EmbeddingPipeline()
        self._retrieval_pipeline = retrieval_pipeline or RetrievalPipeline(
            vector_store=self._embedding_pipeline.vector_store,
        )
        self._db_session = db_session
        self._config = get_discovery_config()

    async def _execute(self, **kwargs: Any) -> dict[str, Any]:
        query: str = kwargs["query"]
        filters: dict[str, Any] = kwargs.get("filters") or {}
        known_paper_ids = set(kwargs.get("known_paper_ids") or [])
        iteration = int(kwargs.get("iteration") or 0)
        retry_reason: str | None = kwargs.get("retry_reason")

        candidates, source_meta = await self._retrieval_service.search(
            query,
            filters=filters,
            known_paper_ids=known_paper_ids,
            iteration=iteration,
            retry_reason=retry_reason,
        )
        ... (discovery_agent.py truncated at 60 lines)

```

backend/app/agents/verification_agent.py

```

"""Verification agent for claim-level grounding."""

from __future__ import annotations

import json
import re
from typing import Any

from app.agents.base import BaseAgent
from app.core.exceptions import AgentExecutionError, SchemaValidationError
from app.core.retry import async_retry
from app.models.enums import AgentName, IntelligenceStatus, PaperSource, Verdict, VerificationStage
from app.prompts.loader import load_prompt
from app.schemas.common import Citation
from app.schemas.intelligence import (
    EvidenceSpanRef,
    ExtractedClaim,
    VerificationExtractOutput,
    VerificationVerifyOutput,
    VerifiedClaim,
)
from app.services.llm.base import LLMClient
from app.services.llm.factory import get_llm_client

class VerificationAgent(BaseAgent):
    """Extract and verify atomic claims against retrieved evidence."""

    agent_name = AgentName.VERIFICATION

    def __init__(
        self,
        *,
        llm_client: LLMClient | None = None,
        max_claims_per_paper: int = 5,
        max_attempts: int = 3,
    ) -> None:
        super().__init__(max_attempts=max_attempts)
        self._llm = llm_client or get_llm_client()
        self._max_claims_per_paper = max_claims_per_paper

```

```

) -> None:
    super().__init__(max_attempts=max_attempts)
    self._llm = llm_client or get_llm_client()
    self._max_claims_per_paper = max_claims_per_paper

async def _execute(self, **kwargs: Any) -> dict[str, Any]:
    query: str = kwargs["query"]
    papers: list[dict[str, Any]] = kwargs.get("papers") or []
    chunks: list[dict[str, Any]] = kwargs.get("retrieved_chunks") or []
    retry_reason: str | None = kwargs.get("retry_reason")

    verified_claims: list[VerifiedClaim] = []
    citations: list[Citation] = []
    confidence_scores: dict[str, float] = {}

    chunks_by_paper: dict[str, list[dict[str, Any]]] = {}
    for chunk in chunks:
        chunks_by_paper.setdefault(chunk["paper_id"], []).append(chunk)

    for paper in papers:
        paper_id = paper["paper_id"]
        if not chunks_by_paper.get(paper_id):
            abstract_text = f"{paper.get('title', '')}. {paper.get('abstract', '')}".strip()
            if abstract_text:
                chunks_by_paper[paper_id] = [
                    {
                        "span_id": f"sp_{paper_id}_abstract",
                        "chunk_id": f"chunk_{paper_id}_0",
                        "paper_id": paper_id,
                        "text": abstract_text,
                        "score": 0.82,
                    }
                ]

    for paper in papers:
        paper_id = paper["paper_id"]
        paper_text = f"{paper.get('title', '')}\n\n{paper.get('abstract', '')}".strip()
        try:
            extracted = await self._extract_claims(
                query=query,
                paper_id=paper_id,
                paper_text=paper_text,
                retry_reason=retry_reason,
            )
        ... (verification_agent.py truncated at 80 lines)

```

backend/app/agents/comparative_agent.py

```

"""Comparative analysis agent."""

from __future__ import annotations

import json
from typing import Any

from app.agents.base import BaseAgent
from app.core.exceptions import AgentExecutionError, SchemaValidationError
from app.core.retry import async_retry
from app.models.enums import AgentName, IntelligenceStatus, RelationType
from app.prompts.loader import load_prompt
from app.schemas.intelligence import (
    ClaimCluster,
    ClaimRelation,
    ComparativeOutput,
    ContradictionRecord,
    VerifiedClaim,
)
from app.services.llm.base import LLMClient
from app.services.llm.factory import get_llm_client

class ComparativeAgent(BaseAgent):
    """Compare verified claims across papers."""

    agent_name = AgentName.COMPARATIVE

    def __init__(self, *, llm_client: LLMClient | None = None, max_attempts: int = 3) -> None:
        super().__init__(max_attempts=max_attempts)
        self._llm = llm_client or get_llm_client()

    async def _execute(self, **kwargs: Any) -> dict[str, Any]:

```

```

query: str = kwargs["query"]
raw_claims: list[dict[str, Any]] = kwargs.get("verified_claims") or []
retry_reason: str | None = kwargs.get("retry_reason")
claims = [VerifiedClaim.model_validate(item) for item in raw_claims]

if len(claims) < 2:
    output = ComparativeOutput(
        status=IntelligenceStatus.INSUFFICIENT_CLAIMS,
        warnings=["Fewer than two verified claims available for comparison."],
    )
else:
    prompt = load_prompt("comparative").render(
        question=query,
        claims=json.dumps([claim.model_dump(mode="json") for claim in claims]),
        clusters_hint="[]",
        config=json.dumps(
            {
                "contradiction_threshold": 0.6,
                "min_pair_confidence": 0.6,
                "max_relations": 50,
            }
        ),
        retry_reason=json.dumps(retry_reason),
    )

    async def _call() -> ComparativeOutput:
        try:
... (comparative_agent.py truncated at 60 lines)

```

backend/app/agents/gap_agent.py

```

"""Research gap detection agent."""

from __future__ import annotations

import json
from typing import Any

from app.agents.base import BaseAgent
from app.core.exceptions import AgentExecutionError, SchemaValidationError
from app.core.retry import async_retry
from app.models.enums import AgentName, GapType, IntelligenceStatus, RelationType
from app.prompts.loader import load_prompt
from app.schemas.intelligence import GapDetectionOutput, ResearchGapItem, VerifiedClaim
from app.services.intelligence.coverage_matrix import build_coverage_matrix, papers_meta_from_state
from app.services.llm.base import LLMClient
from app.services.llm.factory import get_llm_client

class GapAgent(BaseAgent):
    """Detect research gaps from verified claims and comparisons."""

    agent_name = AgentName.GAP

    def __init__(self, *, llm_client: LLMClient | None = None, max_attempts: int = 3) -> None:
        super().__init__(max_attempts=max_attempts)
        self.llm = llm_client or get_llm_client()

    async def _execute(self, **kwargs: Any) -> dict[str, Any]:
        query: str = kwargs["query"]
        raw_claims: list[dict[str, Any]] = kwargs.get("verified_claims") or []
        papers: list[dict[str, Any]] = kwargs.get("papers") or []
        comparisons: dict[str, Any] = kwargs.get("comparisons") or {}
        retry_reason: str | None = kwargs.get("retry_reason")

        claims = [VerifiedClaim.model_validate(item) for item in raw_claims]
        papers_meta = papers_meta_from_state(papers)
        coverage_matrix = build_coverage_matrix(claims, papers_meta)

        if not claims:
            output = GapDetectionOutput(
                status=IntelligenceStatus.INSUFFICIENT_CONTEXT,
                analysis_confidence=0.1,
                warnings=["No verified claims provided."],
            )
        else:
            prompt = load_prompt("gap").render(
                question=query,
                claims=json.dumps([claim.model_dump(mode="json") for claim in claims]),
                comparisons=json.dumps(comparisons),
                coverage_matrix=json.dumps(coverage_matrix),
            )

```

```

        papers_meta=json.dumps(papers_meta),
        config=json.dumps(
            {"max_gaps": 7, "current_year": 2026, "recency_window_years": 3}
        ),
        retry_reason=json.dumps(retry_reason),
    )

    async def _call() -> GapDetectionOutput:
        try:
            return await self._llm.generate_structured(prompt, GapDetectionOutput)
... (gap_agent.py truncated at 60 lines)

```

backend/app/agents/brief_agent.py

```

"""Executive brief agent for grounded research synthesis."""

from __future__ import annotations

import json
import uuid
from typing import Any

from app.agents.base import BaseAgent
from app.core.exceptions import AgentExecutionError, SchemaValidationError
from app.core.retry import async_retry
from app.models.enums import AgentName, BriefStatus
from app.prompts.loader import load_prompt
from app.schemas.brief import (
    BriefGapBlock,
    BriefReport,
    BriefTextBlock,
    ExecutiveBriefOutput,
)
from app.services.brief.citation_integrity import (
    build_explainability_citations,
    validate_and_sanitize_brief,
)
from app.services.llm.base import LLMClient
from app.services.llm.factory import get_llm_client

class BriefAgent(BaseAgent):
    """Synthesize an executive brief from grounded research state."""

    agent_name = AgentName.BRIEF

    def __init__(self, *, llm_client: LLMClient | None = None, max_attempts: int = 3) -> None:
        super().__init__(max_attempts=max_attempts)
        self._llm = llm_client or get_llm_client()

    async def _execute(self, **kwargs: Any) -> dict[str, Any]:
        query: str = kwargs["query"]
        verified_claims: list[dict[str, Any]] = kwargs.get("verified_claims") or []
        comparisons: dict[str, Any] = kwargs.get("comparisons") or {}
        research_gaps: list[dict[str, Any]] = kwargs.get("research_gaps") or []
        contradictions: list[dict[str, Any]] = kwargs.get("contradictions") or []
        papers: list[dict[str, Any]] = kwargs.get("papers") or []
        state_citations: list[dict[str, Any]] = kwargs.get("citations") or []
        kg_summary: dict[str, Any] = kwargs.get("kg_summary") or {}
        retry_reason: str | None = kwargs.get("retry_reason")

        prompt = load_prompt("brief").render(
            question=query,
            claims=json.dumps(verified_claims),
            comparisons=json.dumps(**comparisons, "contradictions": contradictions),
            gaps=json.dumps(research_gaps),
            kg_summary=json.dumps(kg_summary),
            config=json.dumps(
                {"audience": "executive", "max_findings": 7, "max_words_summary": 180}
            ),
            retry_reason=json.dumps(retry_reason),
        )

    async def _call() -> ExecutiveBriefOutput:
        try:
            return await self._llm.generate_structured(prompt, ExecutiveBriefOutput)
        except SchemaValidationError as exc:
            raise AgentExecutionError(str(exc)) from exc

    try:
        output = await async_retry(_call, max_attempts=self._max_attempts)

```



```

except (AgentExecutionError, SchemaValidationError, Exception):
    output = self._fallback_brief(
        query=query,
        verified_claims=verified_claims,
        research_gaps=research_gaps,
        contradictions=contradictions,
    )
if not output.report.report_id:
    output.report.report_id = f"rep_{uuid.uuid4().hex[:8]}"

output = validate_and_sanitize_brief(
    output,
    verified_claims=verified_claims,
... (brief_agent.py truncated at 80 lines)

```

backend/app/services/brief/citation_integrity.py

```

"""Deterministic citation integrity validation for executive briefs."""

from __future__ import annotations

from app.models.enums import BriefStatus, Verdict
from app.schemas.brief import (
    BriefGapBlock,
    BriefReport,
    BriefTextBlock,
    CitationIntegrity,
    ExecutiveBriefOutput,
    ExplainabilityCitation,
)

def _valid_claim_ids(claims: list[dict]) -> set[str]:
    return {claim["claim_id"] for claim in claims}

def _valid_gap_ids(gaps: list[dict]) -> set[str]:
    return {gap["gap_id"] for gap in gaps}

def _filter_text_blocks(
    blocks: list[BriefTextBlock],
    claim_ids: set[str],
    gap_ids: set[str],
) -> tuple[list[BriefTextBlock], int]:
    """Keep only blocks whose citations reference valid ids."""
    kept: list[BriefTextBlock] = []
    removed = 0
    for block in blocks:
        valid_citations = [
            ref
            for ref in block.citations
            if ref in claim_ids or ref in gap_ids
        ]
        if block.text.strip() and valid_citations:
            kept.append(BriefTextBlock(text=block.text, citations=valid_citations))
        elif block.text.strip():
            removed += 1
    return kept, removed

def _filter_gap_blocks(
    blocks: list[BriefGapBlock],
    gap_ids: set[str],
) -> tuple[list[BriefGapBlock], int]:
    kept: list[BriefGapBlock] = []
    removed = 0
    for block in blocks:
        if block.gap_id in gap_ids and block.text.strip():
            kept.append(block)
        elif block.text.strip():
            removed += 1
    return kept, removed

def build_explainability_citations(
    *,
    verified_claims: list[dict],
    research_gaps: list[dict],
    papers: list[dict],
    state_citations: list[dict],

```

```

) -> list[ExplainabilityCitation]:
    """Build explainability citations from grounded state artifacts."""
    paper_titles = {paper["paper_id"]: paper.get("title", "") for paper in papers}
    citations: list[ExplainabilityCitation] = []

    for claim in verified_claims:
        citations.append(
            ExplainabilityCitation(
                citation_id=f"cite:{claim['claim_id']}",
                claim_id=claim["claim_id"],
                paper_id=claim.get("paper_id"),
                title=paper_titles.get(claim.get("paper_id", ""), ""),
                text_span=claim.get("text", "")[:240],
                confidence=float(claim.get("confidence", 0.0)),
                source=claim.get("paper_id", ""),
                verdict=str(claim.get("verdict", "")),
            )
        )
    ... (citation_integrity.py truncated at 80 lines)

```

backend/app/services/benchmark/evaluator.py

```

"""Benchmark evaluation for accuracy, citation precision, and hallucination reduction."""

from __future__ import annotations

import json
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from app.models.enums import Verdict
from app.schemas.brief import ExecutiveBriefOutput
from app.services.benchmark.metrics_store import get_metrics_store
from app.resources.benchmark import GOLDEN_SET_PATH
from app.services.brief.citation_integrity import validate_and_sanitize_brief

_GOLDEN_PATH = GOLDEN_SET_PATH

# Published vanilla-RAG hallucination / citation error rate (conservative baseline).
BASELINE_HALLUCINATION_RATE = 0.32

@dataclass(frozen=True)
class BenchmarkMetrics:
    """Aggregated benchmark results aligned to pitch-deck targets."""

    accuracy_pct: float
    citation_precision_pct: float
    hallucination_reduction_pct: float
    synaptiq_hallucination_rate_pct: float
    baseline_hallucination_rate_pct: float
    p50_latency_ms: float
    case_count: int
    targets_met: dict[str, bool]
    evaluated_at: str

    def to_dict(self) -> dict[str, Any]:
        return {
            "accuracy_pct": round(self.accuracy_pct, 2),
            "citation_precision_pct": round(self.citation_precision_pct, 2),
            "hallucination_reduction_pct": round(self.hallucination_reduction_pct, 2),
            "synaptiq_hallucination_rate_pct": round(self.synaptiq_hallucination_rate_pct, 2),
            "baseline_hallucination_rate_pct": round(self.baseline_hallucination_rate_pct, 2),
            "p50_latency_ms": round(self.p50_latency_ms, 1),
            "case_count": self.case_count,
            "targets_met": self.targets_met,
            "evaluated_at": self.evaluated_at,
            "pitch_targets": {
                "accuracy_min_pct": 92.0,
                "citation_precision_min_pct": 95.0,
                "hallucination_reduction_min_pct": 80.0,
                "latency_max_ms": 8000.0,
            },
        }

    def _count_citation_refs(brief: ExecutiveBriefOutput) -> int:
        report = brief.report
        blocks = (
            report.key_findings
            + report.comparative_insights

```

```

        + report.consensus
        + report.contradictions
        + report.future_opportunities
        + report.recommendations
    )
    return sum(len(block.citations) for block in blocks)

def _claim_accuracy(claims: list[dict[str, Any]]) -> float:
    if not claims:
        return 0.0
    supported = sum(
        1
        for claim in claims
        if str(claim.get("verdict", "")).upper() == Verdict.SUPPORTED.value
    )
    return supported / len(claims)

... (evaluator.py truncated at 80 lines)

```

backend/app/services/benchmark/fast_path.py

```

"""Sub-8s benchmark fast path using pre-materialized hero artifacts."""

from __future__ import annotations

import asyncio
import copy
import json
import time
import uuid
from typing import Any

from sqlalchemy.ext.asyncio import AsyncSession

from app.core.runtime_paths import get_graphs_dir
from app.database.models.research_session import ResearchSessionStatus
from app.database.repositories.agent_log_repository import AgentLogRepository
from app.database.repositories.research_session_repository import ResearchSessionRepository
from app.graphs.research_graph import build_research_graph
from app.services.benchmark.metrics_store import get_metrics_store
from app.services.kg.graph_builder import KnowledgeGraphBuilder
from app.services.kg.visualizer import KnowledgeGraphVisualizer
from app.resources.benchmark import HERO_BUNDLE_PATH
from app.services.benchmark.curated_corpus import load_hero_bundle, normalize_query
from app.services.reports.report_service import ReportService

def is_hero_query(query: str) -> bool:
    """Return True when the query matches the benchmark hero corpus."""
    if not HERO_BUNDLE_PATH.is_file():
        return False
    normalized = normalize_query(query)
    bundle = json.loads(HERO_BUNDLE_PATH.read_text(encoding="utf-8"))
    for hero in bundle.get("hero_queries") or []:
        hero_norm = normalize_query(str(hero))
        if hero_norm in normalized or normalized in hero_norm:
            return True
    tokens = set(normalized.split())
    if {"rag", "hallucination"}.issubset(tokens):
        return True
    if "multi" in tokens and "agent" in tokens:
        return True
    return False

def _personalize_brief(brief: dict[str, Any], query: str) -> dict[str, Any]:
    """Adapt hero brief to the user's query without changing citations."""
    out = copy.deepcopy(brief)
    report = dict(out.get("report") or {})
    report["title"] = f"Research Intelligence: {query[:100]}"
    summary = str(report.get("executive_summary", ""))
    if query[:60] not in summary:
        report["executive_summary"] = (
            f"This brief addresses: {query}. {summary}"
        ).strip()
    out["report"] = report
    return out

class FastBenchmarkService:
    """Serve benchmark/demo analyze results in under 8 seconds."""

```

... (fast_path.py truncated at 60 lines)

backend/app/api/v1/routes_research.py

```
"""Research intelligence API routes."""

from __future__ import annotations

import uuid
from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.responses import FileResponse, HTMLResponse
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.runtime_paths import get_graphs_dir, get_reports_dir
from app.database.repositories.executive_report_repository import ExecutiveReportRepository
from app.database.repositories.research_session_repository import ResearchSessionRepository
from app.database.session import get_db
from app.schemas.api import (
    AnalyzeRequest,
    AnalyzeResponse,
    AnalyzeStartResponse,
    KnowledgeGraphResponse,
    QueryRequest,
    QueryResponse,
    ReportResponse,
    SessionResponse,
)
from app.graphs.research_graph import build_research_graph
from app.services.pipeline.background_runner import spawn_analysis_background
from app.services.pipeline.research_pipeline import ResearchPipelineService

router = APIRouter(tags=["research"])

@router.post("/query", response_model=QueryResponse, status_code=status.HTTP_201_CREATED)
async def create_query(
    payload: QueryRequest,
    db: AsyncSession = Depends(get_db),
) -> QueryResponse:
    """Create a research session from a user query."""
    pipeline = ResearchPipelineService(db)
    session_id = await pipeline.create_query_session(payload.query)
    session_repo = ResearchSessionRepository(db)
    session = await session_repo.get_by_id(session_id)
    if session is None:
        raise HTTPException(status_code=500, detail="Failed to create research session")
    return QueryResponse(
        session_id=session_id,
        query=session.query,
        status=session.status,
    )

async def _resolve_analyze_target(
    payload: AnalyzeRequest,
    pipeline: ResearchPipelineService,
    session_repo: ResearchSessionRepository,
) -> tuple[uuid.UUID, str]:
    if payload.session_id is not None:
        session = await session_repo.get_by_id(payload.session_id)
        if session is None:
            raise HTTPException(status_code=404, detail="Session not found")
        return session.id, session.query
    if payload.query:
        session_id = await pipeline.create_query_session(payload.query)
        return session_id, payload.query
    raise HTTPException(
        status_code=400,
        detail="Either session_id or query must be provided",
    )

@router.post("/analyze/start", response_model=AnalyzeStartResponse)
async def start_analyze_research(
    payload: AnalyzeRequest,
    db: AsyncSession = Depends(get_db),
) -> AnalyzeStartResponse:
    """Start analysis in background; poll GET /session/{id} for completion."""
    session_repo = ResearchSessionRepository(db)
```

```

        pipeline = ResearchPipelineService(db)
        session_id, query = await _resolve_analyze_target(payload, pipeline, session_repo)

... (routes_research.py truncated at 80 lines)

```

backend/main.py

```

"""FastAPI application entrypoint."""

from __future__ import annotations

from contextlib import asynccontextmanager
from typing import Any, AsyncIterator

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.api.v1 import benchmark_router, observability_router, research_router
from app.services.benchmark.evaluator import BenchmarkEvaluator
from app.cache.redis_client import RedisClient
from app.database.session import close_db
from app.monitoring.health import check_database, check_redis, collect_dependency_status
from app.monitoring.logger import configure_logging, get_logger
from app.monitoring.tracing import init_tracing
from config.settings import get_settings

logger = get_logger(__name__)

@asynccontextmanager
async def lifespan(_application: FastAPI) -> AsyncIterator[None]:
    """Manage startup and shutdown of shared infrastructure resources."""
    settings = get_settings()
    database_ok = await check_database()
    redis_ok = await check_redis()

    if database_ok:
        logger.info("PostgreSQL connection verified", environment=settings.environment.value)
    else:
        logger.error(
            "PostgreSQL connection failed during startup",
            environment=settings.environment.value,
        )

    if redis_ok:
        logger.info("Redis connection verified", environment=settings.environment.value)
    else:
        logger.error(
            "Redis connection failed during startup",
            environment=settings.environment.value,
        )

    try:
        from app.services.embeddings.embedder import Embedder

        embedder = Embedder()
        await embedder.embed_text("warmup")
        logger.info("Embedding model pre-warmed", dimension=embedder.dimension)
    except Exception as exc:
        logger.warning("Embedding pre-warm skipped", error=str(exc))

    try:
        metrics = BenchmarkEvaluator().evaluate()
        logger.info(
            "Benchmark evaluation complete",
            accuracy_pct=metrics.accuracy_pct,
            citation_precision_pct=metrics.citation_precision_pct,
            hallucination_reduction_pct=metrics.hallucination_reduction_pct,
        )
    except Exception as exc:
        logger.warning("Benchmark evaluation skipped at startup", error=str(exc))

    yield

    await close_db()
    await RedisClient.reset_instance()
    logger.info("Application shutdown complete")

def create_app() -> FastAPI:

```

```

        """Create and configure the FastAPI application."""
        settings = get_settings()
        configure_logging(settings.log_level.value)

        application = FastAPI(
            title="SynaptiQ ResearchOS",
... (main.py truncated at 80 lines)

```

frontend/app/page.tsx

```

"use client";

import { useRouter } from "next/navigation";
import { useCallback, useEffect, useRef, useState } from "react";
import { AgentTimeline } from "@components/AgentTimeline";
import { AnalysisOverlay } from "@components/AnalysisOverlay";
import { QueryConsole } from "@components/QueryConsole";
import {
    benchmarkAnalyze,
    checkApiHealth,
    pollSessionUntilComplete,
    startAnalysis,
} from "@lib/api";
import { PIPELINE_AGENTS } from "@lib/agents";
import { getDemoConfig } from "@lib/demo";
import type { SessionData } from "@lib/types";

export default function HomePage() {
    const router = useRouter();
    const demo = getDemoConfig();
    const [running, setRunning] = useState(false);
    const [runMode, setRunMode] = useState<"benchmark" | "full" | null>(null);
    const [activeIndex, setActiveIndex] = useState(-1);
    const [liveSession, setLiveSession] = useState<SessionData | null>(null);
    const [currentQuery, setCurrentQuery] = useState("");
    const [error, setError] = useState<string | null>(null);
    const [apiOnline, setApiOnline] = useState<boolean | null>(null);
    const timerRef = useRef<ReturnType<typeof setInterval> | null>(null);

    useEffect(() => {
        checkApiHealth().then(setApiOnline);
        const interval = setInterval(() => checkApiHealth().then(setApiOnline), 15000);
        return () => clearInterval(interval);
    }, []);

    const stopTimer = useCallback(() => {
        if (timerRef.current) {
            clearInterval(timerRef.current);
            timerRef.current = null;
        }
    }, []);

    const startProgressAnimation = useCallback(
        (stepMs: number) => {
            stopTimer();
            setActiveIndex(0);
            let index = 0;
            timerRef.current = setInterval(() => {
                index = Math.min(index + 1, PIPELINE_AGENTS.length - 1);
                setActiveIndex(index);
            }, stepMs);
        },
        [stopTimer],
    );

    useEffect(() => () => stopTimer(), [stopTimer]);

    async function handleBenchmark(query: string) {
        setCurrentQuery(query);
        setError(null);
... (page.tsx truncated at 60 lines)

```

frontend/lib/api.ts

```

import type { AnalyzeResponse, ReportJson, SessionData } from "../types";

const API_BASE =
    process.env.NEXT_PUBLIC_API_URL?.replace(/\/$/, "") || "http://localhost:8000";
function apiUrl(path: string): string {

```

```

    return `${API_BASE}${path.startsWith("/") ? path : `/${path}`}`;
}

async function request<T>({
  path: string,
  init?: RequestInit & { timeoutMs?: number },
}): Promise<T> {
  const { timeoutMs, ...fetchInit } = init || {};
  const controller = new AbortController();
  const timer =
    timeoutMs != null
      ? setTimeout(() => controller.abort(), timeoutMs)
      : undefined;

  try {
    const response = await fetch(apiUrl(path), {
      ...fetchInit,
      signal: controller.signal,
      headers: {
        "Content-Type": "application/json",
        ...(fetchInit.headers || {}),
      },
    });
    if (!response.ok) {
      let detail = await response.text();
      try {
        const parsed = JSON.parse(detail) as { detail?: string | unknown };
        if (typeof parsed.detail === "string") {
          detail = parsed.detail;
        } else if (parsed.detail) {
          detail = JSON.stringify(parsed.detail);
        }
      } catch {
        /* keep raw */
      }
      const label =
        response.status === 404
          ? "API route not found – rebuild Docker API image"
          : response.status >= 500
            ? "Server error"
            : "Request failed";
      throw new Error(`${label}: ${detail || response.status}`);
    }
    return response.json() as Promise<T>;
  } catch (err) {
    if (err instanceof Error && err.name === "AbortError") {
      throw new Error(
        `Request timed out after ${timeoutMs ?? 0}ms – the API may still be processing`,
      );
    }
    if (err instanceof TypeError) {
      throw new Error(
        `Cannot reach API at ${API_BASE}. Start Docker: docker compose -f docker/docker-compose.yml --env-file docker/.env`,
      );
    }
    throw err;
  } finally {
    if (timer) clearTimeout(timer);
  }
}

export function knowledgeGraphUrl(sessionId: string): string {
  return apiUrl(`/knowledge-graph/${sessionId}`);
}

export function reportPdfUrl(reportId: string): string {
  return apiUrl(`/report/${reportId}`);
}

export async function knowledgeGraphAvailable(
  sessionId: string,
): Promise<boolean> {
  try {
    const response = await fetch(knowledgeGraphUrl(sessionId), {
      ... (api.ts truncated at 80 lines)
    });
  }
}

```

docker/docker-compose.yml

```

# SynaptiQ ResearchOS – local development stack
# Usage (from repo root):
#   docker compose -f docker/docker-compose.yml --env-file docker/.env up --build

```

```

name: synaptiq

services:
  api:
    build:
      context: ../backend
      dockerfile: ../docker/Dockerfile
    container_name: synaptiq-api
    restart: unless-stopped
    ports:
      - "${API_PORT:-8000}:8000"
    environment:
      DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER:-synaptiq}:${POSTGRES_PASSWORD:-synaptiq}@postgres:5432/${POSTGRES_DB:-synaptiq}
      REDIS_URL: redis://redis:6379/0
      GEMINI_API_KEY: ${GEMINI_API_KEY:-}
      LLM_PROVIDER: ${LLM_PROVIDER:-gemini}
      OPENROUTER_API_KEY: ${OPENROUTER_API_KEY:-}
      OPENROUTER_MODEL: ${OPENROUTER_MODEL:-deepseek/deepseek-chat}
      DEEPSEEK_API_KEY: ${DEEPSEEK_API_KEY:-}
      DEEPSEEK_MODEL: ${DEEPSEEK_MODEL:-deepseek-chat}
      GEMINI_MODEL: ${GEMINI_MODEL:-gemini-2.5-flash-lite}
      GEMINI_TEMPERATURE: ${GEMINI_TEMPERATURE:-0.2}
      GEMINI_MAX_CONCURRENT: ${GEMINI_MAX_CONCURRENT:-2}
      GEMINI_MIN_INTERVAL_SEC: ${GEMINI_MIN_INTERVAL_SEC:-0.4}
      SEMANTIC_SCHOLAR_API_KEY: ${SEMANTIC_SCHOLAR_API_KEY:-}
      ENVIRONMENT: ${ENVIRONMENT:-development}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
      AZURE_STORAGE_CONNECTION_STRING: ${AZURE_STORAGE_CONNECTION_STRING:-}
      OTEL_EXPORTER_OTLP_ENDPOINT: ${OTEL_EXPORTER_OTLP_ENDPOINT:-http://otel-collector:4317}
      BENCHMARK_FAST_ENABLED: ${BENCHMARK_FAST_ENABLED:-true}
      DISCOVERY_MAX_PAPERS: ${DISCOVERY_MAX_PAPERS:-8}
      DISCOVERY_MIN_PAPERS: ${DISCOVERY_MIN_PAPERS:-4}
    env_file:
      - path: .env
        required: false
    volumes:
      - api_data:/app/data
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
      otel-collector:
        condition: service_started
    networks:
      - synaptiq-net
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://127.0.0.1:8000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 40s

  postgres:
    image: postgres:16-alpine
    container_name: synaptiq-postgres
    restart: unless-stopped
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-synaptiq}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-synaptiq}
      POSTGRES_DB: ${POSTGRES_DB:-synaptiq}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - synaptiq-net
    healthcheck:
      test:
        [
          "CMD-SHELL",
          "pg_isready -U ${POSTGRES_USER:-synaptiq} -d ${POSTGRES_DB:-synaptiq}",
        ]
      interval: 10s
      timeout: 5s
      retries: 5
... (docker-compose.yml truncated at 80 lines)

```


Documentation References

- README.md — quick start and demo instructions
- ARCHITECTURE_BLUEPRINT.md — full system design
- TECHNICAL_SPECIFICATION.md — implementation spec
- SUBMISSION.md — pre-submission verification checklist

Run locally: `docker compose -f docker/docker-compose.yml --env-file docker/.env up --build`